

SQL Complete Reference Guide

Basics | Filtering | Aggregation | Joins | Advanced

Table of Contents

Chapter 1 — Basics *SELECT, ORDER BY, LIMIT*

Chapter 2 — Filtering *WHERE, BETWEEN, IN, LIKE, IS NULL, HAVING*

Chapter 3 — Text & Dates *CONCAT, REPLACE, DATEDIFF, CEIL, FLOOR*

Chapter 4 — Aggregation *SUM, AVG, COUNT, GROUP BY, CASE WHEN*

Chapter 5 — Joins *INNER JOIN, Multi-table joins, Subquery, CTE*

Chapter 6 — Advanced *RANK, DENSE_RANK, Window Functions, LAG, VIEW*

Chapter 1 — Basics

SELECT statements, ORDER BY, LIMIT

These are the building blocks of every SQL query. Master these before moving to anything else.

SELECT Statements

SELECT * (all columns) [easy]

The asterisk (*) means "give me every column". Fine for exploring data quickly, but avoid in production — it fetches columns you may not need, which is slower.

```
SELECT * FROM customers; -- returns every column and every row in the table
```

SELECT specific columns [easy]

Name only the columns you actually need. This is faster, produces cleaner output, and is much easier for others to read.

```
SELECT customer_id, customer_city, customer_state FROM customers; -- only 3 columns returned instead of all columns
```

schema.table notation [easy]

When your server hosts multiple databases, use database_name.table_name so SQL knows exactly which database and table to look in.

```
SELECT * FROM ecommerce.customers; -- ecommerce = database name | customers = table name
```

Sorting & Limiting Results

ORDER BY [easy]

Sorts the result set. Default direction is ASC (lowest to highest). Add DESC to reverse the order. You can sort by multiple columns — the second column sorts ties from the first.

```
SELECT * FROM payments ORDER BY payment_value; -- low to high (ASC)
SELECT * FROM payments ORDER BY payment_value DESC; -- high to low
-- sort by 2 columns: value ASC, then type DESC for ties
SELECT * FROM payments ORDER BY payment_value, payment_type DESC;
```

LIMIT [easy]

Restricts how many rows are returned. Essential when previewing large tables so you do not load millions of rows at once. LIMIT offset, count lets you skip rows — useful for pagination.

```
SELECT * FROM payments LIMIT 5; -- first 5 rows only
SELECT * FROM payments LIMIT 2, 3; -- skip 2 rows, return next 3
-- LIMIT offset, count: used for pagination (skip N, take M)
```

Tip: Think of LIMIT 2,3 like a book: skip chapters 1 and 2, then read 3 chapters starting from chapter 3.

Chapter 2 — Filtering

WHERE, BETWEEN, IN, LIKE, IS NULL, HAVING

Filtering is how you tell SQL "give me only the rows I care about." WHERE filters before grouping. HAVING filters after grouping.

WHERE Conditions

= AND OR NOT [easy]

The core logic operators. = is exact match. AND requires ALL conditions to be true. OR requires ANY ONE condition to be true. NOT flips / excludes the condition.

```
SELECT * FROM customers WHERE customer_state = 'MG'; -- exact match: only rows where state is exactly "MG"
SELECT * FROM customers WHERE customer_state = 'MG' OR customer_state = 'SP'; -- returns MG rows AND SP rows (either condition satisfied)
SELECT * FROM customers WHERE NOT (customer_state = 'MG' OR customer_state = 'SP'); -- excludes both MG and SP, returns everything else -- parentheses group the OR first, then NOT applies to the whole group
```

Multiple AND conditions [easy]

Chain as many AND conditions as needed. Every single condition must be true for a row to appear in the result.

```
SELECT * FROM payments WHERE payment_type = 'credit_card' AND payment_value >= 1000 AND payment_installments = 2; -- all 3 conditions must match simultaneously -- numeric values (1000, 2) need no quotes | text needs quotes
```

BETWEEN [easy]

Filters a range of values. Both endpoints are INCLUDED (>= start AND <= end). Works on numbers and dates.

```
SELECT * FROM payments WHERE payment_value BETWEEN 150 AND 200; -- includes 150, 200, and every value in between (inclusive both ends)
```

IN / NOT IN [easy]

Shortcut for multiple OR conditions on the same column. Much cleaner than writing OR ten times.

```
SELECT * FROM customers WHERE customer_state IN ('SC', 'PR', 'SP', 'MG'); -- same as: state='SC' OR state='PR' OR state='SP' OR state='MG'
SELECT * FROM customers WHERE customer_state NOT IN ('SC', 'PR', 'SP', 'MG'); -- returns all states EXCEPT the 4 listed
```

LIKE (pattern matching) [easy]

Used for partial text matching. The % symbol is a wildcard — it means "any number of any characters here".

```
SELECT * FROM customers WHERE customer_city LIKE 'r%'; -- starts with 'r'
SELECT * FROM customers WHERE customer_city LIKE '%r'; -- ends with 'r'
SELECT * FROM customers WHERE customer_city LIKE '%de%'; -- 'de' anywhere in name
```

Tip: Think of % as a gap-filler: 'r%' = r then anything. '%r' = anything then r. '%de%' = anything, de, anything.

IS NULL [easy]

Finds rows where a column has NO value at all. NULL is not zero or empty string — it means the value is completely absent/unknown.

```
SELECT * FROM orders WHERE order_delivered_customer_date IS NULL; -- orders that were never delivered (delivery date is missing) -- use IS NULL, NOT = NULL (= NULL never works in SQL)
```

HAVING (filter after grouping)

WHERE vs HAVING [intermediate]

WHERE filters individual rows BEFORE they are grouped. HAVING filters group results AFTER aggregation. Use HAVING when your condition involves an aggregate like AVG, SUM, or COUNT.

```
-- WHERE: filters rows before grouping SELECT payment_type, ROUND(AVG(payment_value), 2) FROM payments WHERE payment_value > 10 -- removes rows below 10 FIRST GROUP BY payment_type; -- HAVING: filters groups after aggregation is done SELECT payment_type, ROUND(AVG(payment_value), 2) FROM payments GROUP BY payment_type HAVING AVG(payment_value) >= 100; -- keeps groups where avg >= 100
```

***Tip:** Rule to remember: WHERE = filtering ingredients before cooking. HAVING = tasting the dish after cooking and removing what you do not like.*

Chapter 3 — Text & Dates

CONCAT, REPLACE, TRIM, DATEDIFF, CEIL, FLOOR

SQL has powerful built-in functions to clean, format, and extract information from text and date columns — no Python needed.

Text Functions

LENGTH & TRIM [easy]

LENGTH counts characters in a string. TRIM removes invisible spaces from both ends before counting. Always TRIM before LENGTH for accurate results.

```
SELECT seller_city, LENGTH(seller_city) FROM sellers; -- raw length (may include hidden spaces at start/end)
SELECT seller_city, LENGTH(TRIM(seller_city)) FROM sellers; -- accurate length after removing leading and trailing spaces
```

UPPER & LOWER [easy]

Converts text case. Useful for standardizing data when the same city name is stored inconsistently as "delhi", "Delhi", and "DELHI".

```
SELECT UPPER(seller_city), LOWER(seller_city) FROM sellers; -- UPPER -> "MUMBAI" | LOWER -> "mumbai"
```

REPLACE [easy]

Finds a piece of text inside a string and swaps it with something else. Every occurrence gets replaced, not just the first one.

```
SELECT seller_city, REPLACE(seller_city, 'a', 'i') FROM sellers; -- REPLACE(column, find_this, replace_with_this) -- 'rajkot' becomes 'rijkot' (every 'a' becomes 'i')
```

CONCAT [easy]

Merges multiple columns or strings into one. Perfect for creating full names, full addresses, or readable labels from separate columns.

```
SELECT CONCAT(seller_city, ' - ', seller_state) AS city_state FROM sellers; -- 'mumbai' + ' - ' + 'MH' -> 'mumbai - MH'
SELECT *, CONCAT(seller_city, ' - ', seller_state) AS city_state FROM sellers; -- keeps all original columns AND adds the new merged column at the end
```

Date & Time Functions

DAY / MONTH / YEAR / MONTHNAME / DAYNAME [easy]

Extracts individual parts from a full datetime value. Use these to group by month, filter by year, or display readable date parts.

```
SELECT order_delivered_customer_date, DAY(order_delivered_customer_date), -- number: 1 to 31
DAYNAME(order_delivered_customer_date), -- text: "Monday"
MONTH(order_delivered_customer_date), -- number: 1 to 12
MONTHNAME(order_delivered_customer_date), -- text: "January"
YEAR(order_delivered_customer_date) -- 4-digit year: 2017 FROM orders;
```

DATEDIFF [intermediate]

Calculates the number of days between two dates. Positive result = first date is later. Negative = first date is earlier (late delivery).

```
SELECT DATEDIFF(order_estimated_delivery_date, order_delivered_customer_date) FROM orders; --
DATEDIFF(end_date, start_date) -- positive -> delivered before estimate (good) -- negative ->
delivered after estimate (late)
```

CEIL & FLOOR [easy]

CEIL always rounds UP to the next whole number. FLOOR always rounds DOWN. Neither uses standard rounding rules — they always go in one direction regardless of the decimal.

```
SELECT payment_value, CEIL(payment_value) FROM payments; -- 99.25 -> 100 (always goes UP)
SELECT payment_value, FLOOR(payment_value) FROM payments; -- 99.25 -> 99 (always goes DOWN)
```

Chapter 4 — Aggregation

SUM, AVG, MAX, MIN, COUNT, GROUP BY, CASE WHEN

Aggregate functions collapse many rows into one summary number. Always pair them with GROUP BY when you want a summary per category.

Core Aggregate Functions

SUM / AVG / MAX / MIN / COUNT [easy]

The 5 most-used aggregate functions. Each reduces a column of values into one single result. ROUND() cleans up messy decimals.

```
SELECT ROUND(SUM(payment_value), 2) AS total_revenue FROM payments; -- adds every value in
the column -> total SELECT ROUND(AVG(payment_value), 2) AS avg_payment FROM payments; --
total sum / number of rows -> average SELECT MAX(payment_value), MIN(payment_value) FROM
payments; -- highest and lowest single value in the column SELECT COUNT(customer_id) FROM
customers; -- all rows (with duplicates) SELECT COUNT(DISTINCT customer_id) FROM customers;
-- unique customers only
```

GROUP BY

GROUP BY basics [intermediate]

Groups rows with the same value together, then applies an aggregate to each group separately. Think of it as: "give me a summary per category."

```
SELECT order_status, COUNT(order_status) AS order_count FROM orders GROUP BY order_status
ORDER BY order_count DESC; -- one row per status: delivered=X, canceled=Y, processing=Z
SELECT payment_type, ROUND(AVG(payment_value), 2), ROUND(MAX(payment_value), 2) FROM payments
GROUP BY payment_type; -- average AND max payment separately for each payment type
```

CASE WHEN (conditional labels) [intermediate]

Adds if-else logic as a new column in your results. You define the conditions and SQL assigns a label based on which one matches first. ELSE is the fallback if nothing matches.

```
WITH a AS ( SELECT product_category AS category, SUM(payment_value) AS sales FROM products
JOIN order_items ON products.product_id = order_items.product_id JOIN payments ON
payments.order_id = order_items.order_id GROUP BY category ) SELECT *, CASE WHEN sales <=
5000 THEN 'Low' -- condition 1 checked first WHEN sales >= 100000 THEN 'High' -- condition 2
ELSE 'Medium' -- fallback if no condition matched END AS sale_type -- name of the new column
FROM a;
```

Tip: CASE is like Excel's IF formula but more powerful. Always include ELSE so no row is left without a label.

Chapter 5 — Joins

INNER JOIN, Multi-table joins, Subquery, CTE

Joins let you combine data from two or more tables using a shared column (foreign key). This is one of the most critical SQL skills for real-world projects.

INNER JOIN

JOIN ... ON (basic join) [intermediate]

INNER JOIN returns only rows that have a matching value in BOTH tables. Rows with no match in either table are silently dropped.

```
SELECT customers.customer_id, orders.order_status FROM customers JOIN orders ON
customers.customer_id = orders.customer_id; -- ON = the shared column that links the two
tables (foreign key) -- only customers who have placed at least one order appear -- add WHERE
to filter the combined result SELECT customers.customer_id, orders.order_status FROM
customers JOIN orders ON customers.customer_id = orders.customer_id WHERE order_status =
'canceled'; -- JOIN first -> combined table -> WHERE then filters the result
```

Joining 3 tables [intermediate]

You can chain multiple JOINS. Each JOIN adds one more table to the combined result. Think of it as a chain — each link connects two tables via a shared column.

```
-- 2-table join: orders + payments SELECT YEAR(orders.order_purchase_timestamp) AS years,
ROUND(SUM(payments.payment_value), 2) AS total FROM orders JOIN payments ON orders.order_id =
payments.order_id GROUP BY years; -- 3-table join: products -> order_items -> payments SELECT
products.product_category, SUM(payments.payment_value) AS sales FROM products JOIN
order_items ON products.product_id = order_items.product_id -- link 1 JOIN payments ON
payments.order_id = order_items.order_id -- link 2 GROUP BY products.product_category;
```

Tip: Visualize joins as a chain: Table A -> shared key -> Table B -> shared key -> Table C. Each JOIN adds one more link.

Subquery vs CTE

Subquery (query inside a query) [intermediate]

A query nested inside another query. The inner query runs first, and its result is used by the outer query. Must always have an alias (AS a). Gets hard to read when deeply nested.

```
SELECT category FROM (SELECT product_category AS category, SUM(payment_value) AS sales FROM
products JOIN order_items ON products.product_id = order_items.product_id JOIN payments ON
payments.order_id = order_items.order_id GROUP BY category ORDER BY sales DESC LIMIT 5) AS a;
-- inner query = top 5 categories by sales -- outer query = selects only category name from
that result -- AS a = subquery must always be given an alias name
```

CTE — WITH ... AS (preferred over subquery) [intermediate]

Same result as a subquery but defined at the top with a name. Much easier to read, debug, and reuse. Always prefer CTE over subquery in real projects.

```
WITH a AS ( -- define CTE once at the top
SELECT product_category AS category,
SUM(payment_value) AS sales
FROM products JOIN order_items ON products.product_id =
order_items.product_id JOIN payments ON payments.order_id = order_items.order_id
GROUP BY category
ORDER BY sales DESC LIMIT 5 )
SELECT category FROM a; -- use CTE like a regular table
```

Tip: CTE = "let me define this first, then use it." Subquery = "define it right here inside." Same result. CTE wins on readability always.

Chapter 6 — Advanced

RANK, DENSE_RANK, Moving Average, LAG, VIEW

Window functions are the most powerful SQL feature. Unlike GROUP BY, they calculate values per row WITHOUT collapsing the table — every original row is preserved with the new calculated column added.

Ranking Functions

RANK() vs DENSE_RANK() [advanced]

Both assign a rank to each row based on ORDER BY inside OVER(). The difference is in how they handle tied values. RANK() skips numbers after a tie. DENSE_RANK() never skips.

```
WITH a AS ( SELECT product_category AS category, SUM(payment_value) AS sales FROM products
JOIN order_items ON products.product_id = order_items.product_id JOIN payments ON
payments.order_id = order_items.order_id GROUP BY category ) SELECT category, sales, RANK()
OVER(ORDER BY sales DESC) AS rank_col, DENSE_RANK() OVER(ORDER BY sales DESC) AS
dense_rank_col FROM a;
```

Tip: Tie at position 2? RANK gives: 1,2,2,4 (skips 3). DENSE_RANK gives: 1,2,2,3 (no skip). For "Top 3" queries, use DENSE_RANK.

Window Functions

SUM() OVER — cumulative running total [advanced]

Calculates a running total — each row shows the sum of all previous rows plus the current row. Great for cumulative revenue or growth charts. The table is NOT collapsed unlike GROUP BY.

```
SELECT order_date, sales, SUM(sales) OVER(ORDER BY order_date) AS cumulative_sales FROM
(SELECT DATE(orders.order_purchase_timestamp) AS order_date, SUM(payments.payment_value) AS
sales FROM orders JOIN payments ON orders.order_id = payments.order_id GROUP BY order_date)
AS a; -- each row = that day sales + all previous days combined -- rows are NOT collapsed
(window function preserves every row)
```

AVG() OVER with ROWS BETWEEN — moving average [advanced]

Calculates the average of the current row and a fixed number of preceding rows. Used in time-series analysis to smooth out spikes and show trends.

```
SELECT customer_id, order_purchase_timestamp, payment, AVG(payment) OVER( PARTITION BY
customer_id -- restart calculation per customer ORDER BY order_purchase_timestamp -- order
transactions by time ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- uses: current row + 2 rows
before = 3-point moving average ) AS moving_avg FROM ( SELECT orders.customer_id,
orders.order_purchase_timestamp, payments.payment_value AS payment FROM payments JOIN orders
ON payments.order_id = orders.order_id ) AS a;
```

Tip: PARTITION BY = restart the window calculation for each group (like GROUP BY but without collapsing rows).

LAG() — year-over-year comparison [advanced]

LAG looks at a value from a previous row. Perfect for comparing current period vs last period without needing a self-join.

```
WITH a AS ( SELECT YEAR(orders.order_purchase_timestamp) AS years,
ROUND(SUM(payments.payment_value), 2) AS payment FROM orders JOIN payments ON orders.order_id
= payments.order_id GROUP BY years ORDER BY years ) SELECT years, (payment - LAG(payment, 1)
OVER(ORDER BY years)) -- current - previous year / LAG(payment, 1) OVER(ORDER BY years) * 100
AS yoy_growth_percent FROM a; -- LAG(column, 1) = value from 1 row above (previous year) --
result: % change in revenue compared to last year
```

VIEW — save a query as a virtual table

CREATE VIEW [intermediate]

Saves a query permanently in the database as a virtual table. You can then query it any time just like a regular table. The underlying data stays in the original tables — the view always reflects the latest data automatically.

```
CREATE VIEW prod_cate_sales AS SELECT products.product_category AS category,
SUM(payments.payment_value) AS sales FROM products JOIN order_items ON products.product_id =
order_items.product_id JOIN payments ON payments.order_id = order_items.order_id GROUP BY
category; -- now use it any time like a regular table: SELECT * FROM prod_cate_sales WHERE
sales > 500000; SELECT * FROM prod_cate_sales ORDER BY sales DESC LIMIT 10;
```

Tip: Think of a VIEW as a saved shortcut. Write the complex JOIN once, save it as a VIEW, then reference it by name forever. Updates to source tables reflect automatically.

Quick Reference Card

Concept	Syntax	Use When
SELECT *	SELECT * FROM table	Exploring data
WHERE	WHERE col = value	Filter rows
AND / OR	WHERE a AND b / a OR b	Multiple conditions
BETWEEN	WHERE col BETWEEN x AND y	Range filter
IN	WHERE col IN (a, b, c)	Multiple values
LIKE	WHERE col LIKE 'r%'	Pattern match
IS NULL	WHERE col IS NULL	Missing values
ORDER BY	ORDER BY col DESC	Sort results
LIMIT	LIMIT 5 / LIMIT 2,3	Restrict rows
SUM/AVG/COUNT	SELECT SUM(col) FROM table	Aggregate data
GROUP BY	GROUP BY col	Summary per group
HAVING	GROUP BY col HAVING AVG() > 100	Filter groups
CASE WHEN	CASE WHEN x THEN 'A' ELSE 'B'	Conditional column
JOIN	FROM a JOIN b ON a.id = b.id	Combine tables
CTE	WITH name AS (...) SELECT ...	Named subquery
RANK()	RANK() OVER(ORDER BY col)	Rank rows
DENSE_RANK()	DENSE_RANK() OVER(ORDER BY col)	Rank (no gaps)
LAG()	LAG(col,1) OVER(ORDER BY col)	Previous row value
SUM() OVER	SUM(col) OVER(ORDER BY col)	Cumulative total
CREATE VIEW	CREATE VIEW name AS SELECT ...	Save a query